A CONSTANT-FACTOR LOWER BOUND FOR $H(n)$

GPT-5.4 PRO

Abstract. We construct explicit hypergraphs showing that

$$H(n) \geq \frac{26}{25}\, k_n \qquad \text{for every } n \geq 15,$$

where $k_1 = 1$ and $k_n = \lfloor n/2 \rfloor + k_{\lfloor n/2 \rfloor} + k_{\lfloor (n+1)/2 \rfloor}$. The witnesses have exactly n distinct edges, no isolated vertices, and are generated by a pure-Python recursive constructor with no nonstandard dependencies. The proof has two ingredients: a substitution theorem for support gadgets, and a finite bootstrap built from exact small frames, seven explicit higher-arity boosters, and a balanced four-way family. All finite checks are reduced to exhaustive verification of explicit inequalities by an archived standalone verifier, and every gadget used by the constructor is written out in the paper. We also record a separate Lubell-family argument giving the asymptotic lower bound $\liminf_{n\to\infty} H(n)/k_n \geq 2\ln 2$, together with an explicit table of the harmonic multiplier γ_t for small t and a self-contained proof that

$$k_n = \frac{1}{2} n \log_2 n + O(n).$$

1. Introduction

Let $G = (V, \mathcal{H})$ be a finite hypergraph. We say that G *contains a partition of size* m if there are $D \subseteq V$ and $\mathcal{P} \subseteq \mathcal{H}$ with $|D| = m$ such that every vertex of D belongs to exactly one edge of $\mathcal{P}$. Write $H(n)$ for the largest $k \in \mathbb{N}$ such that there exists a hypergraph $(V, \mathcal{H})$ with $|V| = k$, no isolated vertices, and no partition of size greater than n. This extremal function arises in Brian and Larson's finitary reformulation of a problem about incompatible ideals [1], and it appears verbatim as an open FrontierMath challenge [2].

Brian and Larson proved the recursive lower bound

$$H(n) \geq k_n, \qquad k_1 = 1, \qquad k_n = \lfloor n/2 \rfloor + k_{\lfloor n/2 \rfloor} + k_{\lfloor (n+1)/2 \rfloor} \quad (n \geq 2).$$

The problem posed on the FrontierMath page asks for a constant-factor improvement on this benchmark, already visible at $n = 15$, and asks for an explicit algorithm that outputs a witness hypergraph for each n [2]. The purpose of this paper is to provide such a construction.

The paper combines the best features of two complementary approaches. The first is a clean *support-gadget substitution theorem*, which turns admissible set systems on a small meta-ground-set into recursive hypergraph constructions. The second is a *finite bootstrap*: below 60 edges we use exact small frames on at most five blocks, together with seven explicit higher-arity boosters; from 60 onward a balanced four-way family propagates a uniform multiplicative gain forever. The official verifier-facing constructor is standard-library Python only: the finitely many exact small frames needed below 60 are precomputed once and for all, and then hard-coded into the final script. All finite computational claims used in Section 3 are verified by the ancillary script `verify_hn_26_25.py`, included with the source and reproduced in Appendix E.

Our principal theorem is the following.

Theorem 1.1. *There exists an explicit sequence of hypergraphs $(G_n)_{n\geq 1}$ with the following properties. For every $n \geq 1$, the hypergraph G_n has exactly n distinct edges, has no isolated vertices, and contains no partition of size greater than n. Moreover, for every $n \geq 15$,*

$$|V(G_n)| \geq \frac{26}{25}\, k_n.$$

Date: March 10, 2026.

2020 *Mathematics Subject Classification.* 05C65, 05D10.

Key words and phrases. hypergraph, exact cover, extremal construction, computer-assisted proof.

Consequently,

$$H(n) \geq \frac{26}{25} k_n \qquad (n \geq 15).$$

The uniform factor $26/25$ *is already in effect at* $n = 15$*, and the minimum ratio in the finite bootstrap window* $15 \leq n < 60$ *is attained at* $n = 17$*:*

$$|V(G_{15})| = 45 > 43 = k_{15}, \qquad |V(G_{17})| = 52 = \frac{26}{25} k_{17}.$$

The construction proving Theorem 1.1 is the one implemented by the function `solution(n)` in Appendix D. It uses only the explicit tables in Appendices A, B, and C. No optimization routine is invoked by the submitted program.

The support-gadget calculus also yields a separate asymptotic statement. This second family is not the official verifier-facing one, but it gives a clean conceptual explanation for the asymptotic constant $2 \ln 2$.

Theorem 1.2. *For every fixed* $t \geq 2$ *there is an explicit recursive family of hypergraphs showing that*

$$H(n) \geq \frac{h_t - 1}{\log_2 t} n \log_2 n - O_t(n), \qquad h_t := 1 + \frac{1}{2} + \cdots + \frac{1}{t}.$$

Consequently,

$$\liminf_{n \to \infty} \frac{H(n)}{k_n} \geq 2 \ln 2.$$

The remainder of the paper is organized as follows. Section 2 develops the general support-gadget substitution theorem. Section 3 builds the explicit $26/25$ family and proves Theorem 1.1. Section 4 records the Lubell-family asymptotics and proves Theorem 1.2, including an explicit table of the harmonic multipliers and a self-contained proof that $k_n = \frac{1}{2} n \log_2 n + O(n)$. The appendices list the exact small frames, the boosters, the four-way residue gadgets, the finite bootstrap table, the official Python constructor, and the ancillary verifier.

2. Support gadgets and substitution

For a hypergraph $G = (V, \mathcal{H})$ and a chosen edge family $\mathcal{P} \subseteq \mathcal{H}$, define

$$u_G(\mathcal{P}) := |\{v \in V : v \text{ belongs to exactly one edge of } \mathcal{P}\}|.$$

The partition problem is equivalent to a uniform bound on u_G.

Lemma 2.1. *A hypergraph* G *contains no partition of size greater than* n *if and only if*

$$u_G(\mathcal{P}) \leq n \qquad \text{for every } \mathcal{P} \subseteq \mathcal{H}.$$

Proof. If $(D, \mathcal{P})$ is a partition of size m, then every vertex of D is counted by $u_G(\mathcal{P})$, so $u_G(\mathcal{P}) \geq m$. Conversely, if $u_G(\mathcal{P}) > n$, let D be the set of vertices counted by $u_G(\mathcal{P})$; then $(D, \mathcal{P})$ is a partition of size $u_G(\mathcal{P})$. □

Fix $t \geq 2$. A *support pattern* on $[t] := \{1, \ldots, t\}$ is a subset of $[t]$ of size at least 2. A finite multiset of support patterns will be denoted by $\mathcal{F}$. Its members will encode new vertices that meet entire blocks of edges simultaneously.

Definition 2.2. Let $\mathbf{n} = (n_1, \ldots, n_t) \in \mathbb{Z}^t_{\geq 0}$. For $I \subseteq T \subseteq [t]$, define

$$\omega_{\mathcal{F}}(T, I) := |\{S \in \mathcal{F} : \ S \subseteq T \text{ and } |S \cap I| = 1\}|,$$

counting multiplicity. We say that $\mathcal{F}$ is an $\mathbf{n}$-*frame* if

$$\omega_{\mathcal{F}}(T, I) \leq \sum_{j \in T \setminus I} n_j \qquad \text{for every } I \subseteq T \subseteq [t].$$

Remark 2.3. Capacity vectors are allowed to have zero coordinates. This convention is needed later for the residue gadgets R_0 and R_1, whose stated capacity vectors contain zeros.

Definition 2.4. Let $\mathcal{F}$ be a support multiset on $[t]$. Let $G_i = (V_i, \mathcal{H}_i)$, $1 \le i \le t$, be hypergraphs with pairwise disjoint vertex sets and pairwise disjoint edge sets. The *substitution hypergraph*

$$\mathcal{F}[G_1, \dots, G_t]$$

has edge set $\mathcal{H}_1 \sqcup \cdots \sqcup \mathcal{H}_t$, and its vertex set consists of $V_1 \sqcup \cdots \sqcup V_t$ together with one new vertex x_S for each occurrence of a support pattern $S \in \mathcal{F}$. The new vertex x_S is incident to every edge of $\mathcal{H}_i$ for every $i \in S$.

Theorem 2.5 (substitution theorem)**.** *Let $\mathbf{n} = (n_1, \dots, n_t) \in \mathbb{Z}^t_{\ge 0}$, and let $\mathcal{F}$ be an $\mathbf{n}$-frame on $[t]$. Assume that each G_i contains no partition of size greater than n_i. Then the substituted hypergraph*

$$G \coloneqq \mathcal{F}[G_1, \dots, G_t]$$

contains no partition of size greater than $n_1 + \cdots + n_t$. Equivalently,

$$u_G(\mathcal{P}) \le n_1 + \cdots + n_t \qquad \text{for every } \mathcal{P} \subseteq \mathcal{E}(G).$$

Proof. Fix $\mathcal{P} \subseteq \mathcal{E}(G)$, and write $\mathcal{P}_i \coloneqq \mathcal{P} \cap \mathcal{H}_i$. Define

$$T \coloneqq \{i : |\mathcal{P}_i| \le 1\}, \qquad I \coloneqq \{i : |\mathcal{P}_i| = 1\}.$$

If $i \notin T$, then $|\mathcal{P}_i| \ge 2$, and the vertices of V_i counted by $u_G(\mathcal{P})$ are counted by $u_{G_i}(\mathcal{P}_i)$, hence contribute at most n_i. If $i \in T \setminus I$, then $|\mathcal{P}_i| = 0$, so the block V_i contributes nothing. If $i \in I$, then again block V_i contributes at most n_i. So the total contribution from the old block vertices is at most

$$\sum_{i \notin T} n_i + \sum_{i \in I} n_i.$$

Now consider a new support vertex x_S. It lies in exactly one selected edge if and only if $S \subseteq T$ and $|S \cap I| = 1$. Hence the total contribution from support vertices is exactly

$$\omega_{\mathcal{F}}(T, I) \le \sum_{j \in T \setminus I} n_j,$$

because $\mathcal{F}$ is an $\mathbf{n}$-frame. Adding the two estimates gives

$$u_G(\mathcal{P}) \le \sum_{i \notin T} n_i + \sum_{i \in I} n_i + \sum_{j \in T \setminus I} n_j = \sum_{i=1}^{t} n_i.$$

Lemma 2.1 finishes the proof. □

Corollary 2.6. *Suppose that for $1 \le i \le t$ there is a hypergraph G_i with exactly n_i edges, no isolated vertices, no partition of size greater than n_i, and a_i vertices. If $\mathcal{F}$ is an $(n_1, \dots, n_t)$-frame, then $\mathcal{F}[G_1, \dots, G_t]$ has exactly $n_1 + \cdots + n_t$ edges, no isolated vertices, no partition of size greater than $n_1 + \cdots + n_t$, and*

$$a_1 + \cdots + a_t + |\mathcal{F}|$$

vertices.

Proof. The edge set is the disjoint union of the edge sets of the blocks, so the edge count is additive. Every old vertex remains non-isolated, and every new support vertex meets every edge in at least one block from its support. The vertex count is immediate, and the partition bound is Theorem 2.5. □

Lemma 2.7. *If $\mathcal{F}$ is an $\mathbf{n}$-frame and $\mathcal{G}$ is an $\mathbf{m}$-frame on the same ground set $[t]$, then $\mathcal{F} \uplus \mathcal{G}$ is an $(\mathbf{n} + \mathbf{m})$-frame.*

Proof. For every $I \subseteq T \subseteq [t]$,

$$\omega_{\mathcal{F} \uplus \mathcal{G}}(T, I) = \omega_{\mathcal{F}}(T, I) + \omega_{\mathcal{G}}(T, I) \le \sum_{j \in T \setminus I} (n_j + m_j).$$

□

Remark 2.8. The entire problem is thus reduced to constructing useful support frames. The strong 26/25 theorem below uses a finite bank of exact frames and boosters. The asymptotic theorem later uses the Lubell family.

3. The uniform factor 26/25

We now describe the explicit family proving Theorem 1.1. There are three ingredients.

First, there is a finite bank of exact small frames on at most five blocks. These were obtained by solving the corresponding integer programs once and for all, and then frozen as explicit support multisets. They are listed in Appendix A.

Second, there are seven higher-arity boosters:

$$B_{15},\ B_{16},\ B_{17},\ B_{19},\ B_{31},\ B_{32},\ B_{33}.$$

These are listed explicitly in Appendix B.

Third, there is the balanced four-way family, consisting of the classical 13-vertex core

$$Q_0 \coloneqq 12 + 13 + 14 + 23 + 24 + 34 + 123 + 124 + 134 + 234 + 3 \cdot 1234$$

on capacity vector $(3, 3, 3, 3)$, together with the twelve residue gadgets R_r, $0 \le r \le 11$, from Appendix B. The core Q_0 is exactly the Lubell frame $\mathcal{L}_4$, since $M_4 = 3$. Hence Proposition 4.1 shows that Q_0 is a $(3, 3, 3, 3)$-frame. As in the appendices, we abbreviate a support pattern $\{i_1, \dots, i_s\}$ by the concatenated string $i_1 \cdots i_s$, and coefficients denote multiplicity.

Proposition 3.1. *Every exact small frame listed in Appendix A, every booster listed in Appendix B, and every residue gadget R_r listed in Appendix B is a valid frame for its stated capacity vector.*

Proof. This is a finite exhaustive check.

For a support multiset $\mathcal{F}$ on t blocks with capacity vector $\mathbf{n} = (n_1, \dots, n_t)$, frame-validity means checking

$$\omega_{\mathcal{F}}(T, I) \le \sum_{j \in T \setminus I} n_j \qquad \text{for every } I \subseteq T \subseteq [t].$$

There are exactly

$$\sum_{T \subseteq [t]} 2^{|T|} = 3^t$$

such pairs (T, I).

The standalone script `verify_hn_26_25.py` in Appendix E exhaustively evaluates these inequalities for every exact frame in Appendix A, every booster in Appendix B, and every residue gadget R_r in Appendix B. The largest gadget used in the uniform construction is B_{33}, which lives on 9 blocks, so the largest exhaustive check has $3^9 = 19683$ cases. Running the verifier confirms that every listed gadget is a valid frame for its stated capacity vector. □

Definition 3.2. Define integers A_n recursively as follows. Set $A_1 \coloneqq 1$. For $2 \le n < 60$, let A_n be the value listed in Appendix C. For $n \ge 60$, write

$$n = 4m + r, \qquad 0 \le r \le 3,$$

and let (a, b, c, d) be the nondecreasing balanced partition of n into four positive parts:

$$(a, b, c, d) = \begin{cases} (m, m, m, m), & r = 0, \\ (m, m, m, m+1), & r = 1, \\ (m, m, m+1, m+1), & r = 2, \\ (m, m+1, m+1, m+1), & r = 3. \end{cases}$$

Then define

$$A_n \coloneqq A_a + A_b + A_c + A_d + e_r(m),$$

where

$$e_0(m) := \left\lfloor \frac{13m}{3} \right\rfloor, \tag{3.1}$$

$$e_1(m) := \left\lfloor \frac{13m+1}{3} \right\rfloor, \tag{3.2}$$

$$e_2(m) := \left\lfloor \frac{13m+5}{3} \right\rfloor, \tag{3.3}$$

$$e_3(m) := \left\lfloor \frac{13m+6}{3} \right\rfloor. \tag{3.4}$$

The formulas (3.1)–(3.4) are exactly the bonuses produced by $\lfloor n/12 \rfloor$ copies of the four-way core Q_0 together with the appropriate residue gadget $R_{n \bmod 12}$.

Theorem 3.3. *For every $n \geq 1$ there exists a hypergraph G_n with exactly n distinct edges, no isolated vertices, no partition of size greater than n, and*

$$|V(G_n)| = A_n.$$

The construction is explicit and recursive.

Proof. We argue by induction on n.

For $n = 1$, take a single edge containing a single vertex.

Assume the statement proved for all smaller positive integers. If $2 \leq n < 60$, the corresponding row of Appendix C specifies one of four possibilities.

Binary case. If the row is $\mathrm{Bin}(a, b)$, then $a + b = n$, the support multiset is $\min(a, b)$ copies of the support 12, and Corollary 2.6 applied to G_a and G_b gives a witness with

$$A_a + A_b + \min(a, b) = A_n$$

vertices.

Exact-frame case. If the row is $F_{(n_1,\ldots,n_t)}$, then the corresponding support multiset is the one listed for that capacity vector in Appendix A. By Proposition 3.1 it is a valid frame, so Corollary 2.6 applied to the smaller witnesses $G_{n_1}, \ldots, G_{n_t}$ yields a witness with

$$A_{n_1} + \cdots + A_{n_t} + \left|F_{(n_1,\ldots,n_t)}\right| = A_n$$

vertices.

Booster case. If the row is B_{15}, B_{16}, B_{17}, B_{19}, B_{31}, B_{32}, or B_{33}, the same argument applies using the explicitly listed support multiset from Appendix B.

Balanced four-way case. If the row is $Q(a, b, c, d)$, let $q := \lfloor n/12 \rfloor$ and $s := n \bmod 12$. The gadget Q_0 is a $(3, 3, 3, 3)$-frame, and R_s is a valid frame for its residual capacity vector by Proposition 3.1. By Lemma 2.7, q copies of Q_0 together with R_s form a frame on capacities (a, b, c, d). Corollary 2.6 then yields a witness with

$$A_a + A_b + A_c + A_d + |q \cdot Q_0 \uplus R_s| = A_n$$

vertices.

Thus the induction closes for all $n < 60$.

Now assume $n \geq 60$. Write $n = 4m + r$ and let (a, b, c, d) be the balanced four-way split. By the inductive hypothesis we have witnesses G_a, G_b, G_c, G_d with the stated properties. Let $q := \lfloor n/12 \rfloor$ and $s := n \bmod 12$. As above, q copies of Q_0 together with R_s form a valid frame on capacities (a, b, c, d). Corollary 2.6 therefore yields a witness with

$$A_a + A_b + A_c + A_d + |q \cdot Q_0 \uplus R_s| = A_a + A_b + A_c + A_d + e_r(m) = A_n$$

vertices. This completes the induction. □

To compare A_n with the benchmark k_n, we need the four-way identities satisfied by k_n.

Lemma 3.4. *For every $m \geq 1$,*

$$\begin{aligned} k_{4m} &= 4k_m + 4m, \\ k_{4m+1} &= 3k_m + k_{m+1} + 4m, \\ k_{4m+2} &= 2k_m + 2k_{m+1} + 4m + 1, \\ k_{4m+3} &= k_m + 3k_{m+1} + 4m + 2. \end{aligned}$$

Proof. The defining recurrence for k_n immediately gives

$$k_{2m} = m + 2k_m, \qquad k_{2m+1} = m + k_m + k_{m+1}.$$

Applying these identities once more gives the displayed four-way formulas. □

Lemma 3.5. *For every integer $m \geq 15$,*

$$\begin{aligned} e_0(m) &\geq \frac{26}{25} \cdot 4m, \\ e_1(m) &\geq \frac{26}{25} \cdot 4m, \\ e_2(m) &\geq \frac{26}{25}(4m+1), \\ e_3(m) &\geq \frac{26}{25}(4m+2). \end{aligned}$$

Proof. Using $\lfloor x \rfloor \geq x - 1$, we obtain

$$\begin{aligned} e_0(m) - \frac{26}{25} \cdot 4m &\geq \frac{13m}{3} - 1 - \frac{104m}{25} = \frac{13m - 75}{75}, \\ e_1(m) - \frac{26}{25} \cdot 4m &\geq \frac{13m+1}{3} - 1 - \frac{104m}{25} = \frac{13m - 50}{75}, \\ e_2(m) - \frac{26}{25}(4m+1) &\geq \frac{13m+5}{3} - 1 - \frac{104m+26}{25} = \frac{13m - 28}{75}, \\ e_3(m) - \frac{26}{25}(4m+2) &\geq \frac{13m+6}{3} - 1 - \frac{104m+52}{25} = \frac{13m - 81}{75}. \end{aligned}$$

All four right-hand sides are affine functions of m with positive slope $13/75$, and at $m = 15$ they equal

$$\frac{120}{75}, \qquad \frac{145}{75}, \qquad \frac{167}{75}, \qquad \frac{114}{75},$$

respectively; hence they are nonnegative for every $m \geq 15$. □

Proposition 3.6. *For every integer n with $15 \leq n < 60$,*

$$A_n \geq \frac{26}{25} k_n.$$

More precisely, Appendix C lists the exact slack

$$\Delta_n := 25A_n - 26k_n,$$

and $\Delta_n \geq 0$ for every $15 \leq n < 60$, with equality only at $n = 17$.

Proof. For each $15 \leq n < 60$, the ancillary verifier in Appendix E checks that the construction listed in Appendix C yields the stated value A_n, and also checks the inequality $25A_n \geq 26k_n$. Since Δ_n is defined by

$$\Delta_n := 25A_n - 26k_n,$$

the printed slack values follow by arithmetic. □

Theorem 3.7. *For every integer $n \geq 15$,*

$$A_n \geq \frac{26}{25} k_n.$$

Hence Theorem 1.1 holds.

Proof. The range $15 \le n < 60$ is Proposition 3.6. Now let $n \ge 60$, write $n = 4m + r$ with $0 \le r \le 3$, and note that $m \ge 15$.

If $r = 0$, then by the definition of A_n, Lemma 3.5, and Lemma 3.4,

$$A_{4m} = 4A_m + e_0(m) \ge \frac{26}{25}(4k_m + 4m) = \frac{26}{25}k_{4m}.$$

If $r = 1$, then

$$A_{4m+1} = 3A_m + A_{m+1} + e_1(m) \ge \frac{26}{25}(3k_m + k_{m+1} + 4m) = \frac{26}{25}k_{4m+1}.$$

If $r = 2$, then

$$A_{4m+2} = 2A_m + 2A_{m+1} + e_2(m) \ge \frac{26}{25}(2k_m + 2k_{m+1} + 4m + 1) = \frac{26}{25}k_{4m+2}.$$

If $r = 3$, then

$$A_{4m+3} = A_m + 3A_{m+1} + e_3(m) \ge \frac{26}{25}(k_m + 3k_{m+1} + 4m + 2) = \frac{26}{25}k_{4m+3}.$$

This completes the induction. □

4. Lubell frames and asymptotic context

We now record the general Lubell family. This family is conceptually cleaner than the finite bootstrap and explains the asymptotic constant $2\ln 2$, although it does not by itself produce the stronger uniform factor $26/25$ already at $n = 15$.

For $t \ge 2$, define

$$M_t := \operatorname{lcm}\left\{\binom{t-1}{r} : 1 \le r \le t-1\right\}.$$

Let $\mathcal{L}_t$ be the support multiset on $[t]$ that contains every j-subset of $[t]$ with multiplicity

$$\frac{M_t}{\binom{t-1}{j-1}} \qquad (2 \le j \le t).$$

Equivalently,

$$\mathcal{L}_t := \biguplus_{j=2}^{t} \frac{M_t}{\binom{t-1}{j-1}} \cdot \binom{[t]}{j}.$$

Proposition 4.1. *For every $t \ge 2$, the support multiset $\mathcal{L}_t$ is an $(M_t, \dots, M_t)$-frame. Moreover,*

$$|\mathcal{L}_t| = M_t\, t\,(h_t - 1), \qquad h_t := 1 + \frac{1}{2} + \cdots + \frac{1}{t}.$$

Proof. Fix $I \subseteq T \subseteq [t]$, and write

$$i := |I|, \qquad z := |T \setminus I|.$$

If $i = 0$ or $z = 0$, then $\omega_{\mathcal{L}_t}(T, I) = 0$, so there is nothing to prove. Assume $i \ge 1$ and $z \ge 1$.

A support pattern of size $r + 1$ contributes to $\omega_{\mathcal{L}_t}(T, I)$ exactly when it chooses one element of I and r elements of $T \setminus I$. Therefore

$$\omega_{\mathcal{L}_t}(T, I) = iM_t \sum_{r=1}^{z} \frac{\binom{z}{r}}{\binom{t-1}{r}}.$$

Use the beta-integral identity

$$\frac{1}{\binom{t-1}{r}} = t \int_0^1 x^r (1-x)^{t-1-r}\, dx.$$

Then

$$\begin{aligned}\sum_{r=0}^{z}\frac{\binom{z}{r}}{\binom{t-1}{r}} &= t\int_0^1\sum_{r=0}^{z}\binom{z}{r}x^r(1-x)^{t-1-r}\,dx\\ &= t\int_0^1(1-x)^{t-1-z}\,dx\\ &= \frac{t}{t-z}.\end{aligned}$$

Subtracting the $r=0$ term gives

$$\sum_{r=1}^{z}\frac{\binom{z}{r}}{\binom{t-1}{r}} = \frac{z}{t-z}.$$

Hence

$$\omega_{\mathcal{L}_t}(T,I) = iM_t\frac{z}{t-z}.$$

Since $i+z=|T|\le t$, we have $i\le t-z$, so

$$\omega_{\mathcal{L}_t}(T,I)\le zM_t = \sum_{j\in T\setminus I}M_t.$$

Thus $\mathcal{L}_t$ is an $(M_t,\dots,M_t)$-frame.

For the size,

$$|\mathcal{L}_t| = \sum_{j=2}^{t}\binom{t}{j}\frac{M_t}{\binom{t-1}{j-1}} = M_t\sum_{j=2}^{t}\frac{t}{j} = M_t\,t\,(h_t-1).$$

□

The case $t=3$ is the exact frame $F_{(2,2,2)}$ from Appendix A; the case $t=4$ is the classical four-way core Q_0 used in the uniform construction. Thus the Lubell family unifies the clean asymptotic side with the explicit uniform one.

Table 1 records the values of γ_t for small t.

t	M_t	$\lvert\mathcal{L}_t\rvert$	$\gamma_t = 2(h_t-1)/\log_2 t$	approx.
2	1	1	1	1.000000
3	2	5	$\frac{5}{3\log_2 3}$	1.051550
4	3	13	$\frac{13}{12}$	1.083333
5	12	77	$\frac{77}{30\log_2 5}$	1.105403
6	10	87	$\frac{29}{10\log_2 6}$	1.121873
7	60	669	$\frac{223}{70\log_2 7}$	1.134774
∞	—	—	$2\ln 2$	1.386294

TABLE 1. Harmonic gadget parameters and asymptotic multipliers γ_t.

Remark 4.2. Already at $t=6$ the Lubell gadget achieves $\gamma_6\approx 1.1219$, which strictly exceeds $\gamma_4 = 13/12\approx 1.0833$. So while the uniform $26/25$ construction in Section 3 uses the four-way core ($t=4$) because it needs explicit control at every $n\ge 15$, the asymptotic picture is strictly better with $t\ge 6$, and the limiting constant is $2\ln 2$.

The $t=6$ Lubell gadget is worth recording explicitly. Here $M_6 = \mathrm{lcm}(1,5,10,10,5,1) = 10$, so $\lambda_r = 10/\binom{5}{r}$ for each $r=1,\dots,5$:

$$\lambda_1 = \tfrac{10}{5} = 2,\quad \lambda_2 = \tfrac{10}{10} = 1,\quad \lambda_3 = \tfrac{10}{10} = 1,\quad \lambda_4 = \tfrac{10}{5} = 2,\quad \lambda_5 = \tfrac{10}{1} = 10.$$

Thus $\mathcal{L}_6$ consists of:

- 2 copies of every 2-element subset of [6] (15 patterns, 30 total),
- 1 copy of every 3-element subset (20 patterns),
- 1 copy of every 4-element subset (15 patterns),

- 2 copies of every 5-element subset (6 patterns, 12 total),
- 10 copies of the full 6-set (10 total).

The total is $30 + 20 + 15 + 12 + 10 = 87 = 6 \cdot 10 \cdot (h_6 - 1)$. The gadget is $(10, 10, 10, 10, 10, 10)$-admissible by Proposition 4.1.

Proposition 4.3. *Let $s_2(m)$ denote the sum of the binary digits of m, with $s_2(0) = 0$. Then for every $n \geq 1$,*

$$k_n = n + \sum_{j=0}^{n-1} s_2(j).$$

Moreover,

$$k_n = \frac{1}{2} n \log_2 n + O(n), \qquad \text{and therefore} \qquad \lim_{n\to\infty} \frac{k_n}{n \log_2 n} = \frac{1}{2}.$$

Proof. Define

$$f(n) := \sum_{j=0}^{n-1} s_2(j), \qquad a_n := n + f(n).$$

Using

$$s_2(2j) = s_2(j), \qquad s_2(2j+1) = s_2(j) + 1,$$

we obtain

$$f(2m) = \sum_{j=0}^{m-1} \big(s_2(2j) + s_2(2j+1)\big) = 2f(m) + m.$$

Hence

$$a_{2m} = 2m + f(2m) = m + 2a_m.$$

Also

$$a_{m+1} = a_m + 1 + s_2(m),$$

so

$$a_{2m+1} = a_{2m} + 1 + s_2(2m) = m + 2a_m + 1 + s_2(m) = m + a_m + a_{m+1}.$$

Thus (a_n) satisfies the same recurrence and initial condition as (k_n):

$$a_1 = 1, \qquad a_{2m} = m + 2a_m, \qquad a_{2m+1} = m + a_m + a_{m+1}.$$

By induction, $a_n = k_n$ for all n, proving the exact formula.

We now prove the asymptotic. Let

$$f(n) := \sum_{j=0}^{n-1} s_2(j), \qquad \text{so } k_n = n + f(n).$$

For each bit position $i \geq 0$, let $w_i(n)$ be the number of integers $0 \leq j < n$ whose i-th binary digit equals 1. Then

$$f(n) = \sum_{i\geq 0} w_i(n),$$

and only the indices $0 \leq i \leq \lfloor \log_2 n \rfloor$ can contribute.

Fix $i \geq 0$ and write $n = q \cdot 2^{i+1} + r$ with $0 \leq r < 2^{i+1}$. In each full block of length 2^{i+1} the i-th digit is 1 for exactly 2^i values, and in the final partial block it contributes $\max(0, r - 2^i)$ ones. Hence

$$w_i(n) = q \cdot 2^i + \max(0, r - 2^i).$$

Using $q = \lfloor n/2^{i+1} \rfloor$, we have

$$\frac{n}{2} - 2^i \leq q \cdot 2^i \leq \frac{n}{2}, \qquad 0 \leq \max(0, r - 2^i) \leq 2^i,$$

so for every $0 \leq i \leq L := \lfloor \log_2 n \rfloor$,

$$\left| w_i(n) - \tfrac{n}{2} \right| \leq 2^i.$$

Summing over $i = 0, 1, \dots, L$ gives

$$\left| f(n) - \frac{(L+1)n}{2} \right| \le \sum_{i=0}^{L} 2^i = 2^{L+1} - 1 < 2n,$$

because $2^L \le n < 2^{L+1}$. Therefore

$$f(n) = \frac{(L+1)n}{2} + O(n).$$

Since $L + 1 = \log_2 n + O(1)$, we conclude

$$f(n) = \frac{1}{2} n \log_2 n + O(n), \qquad k_n = n + f(n) = \frac{1}{2} n \log_2 n + O(n),$$

and hence

$$\lim_{n\to\infty} \frac{k_n}{n \log_2 n} = \frac{1}{2}.$$

□

Theorem 4.4. *Fix $t \ge 2$. There exists a constant $C_t > 0$ such that*

$$H(n) \ge \frac{h_t - 1}{\log_2 t} n \log_2 n - C_t n \qquad \textit{for all } n \ge 1.$$

Proof. Fix $t \ge 2$, and write

$$M := M_t, \qquad B := |\mathcal{L}_t| = Mt(h_t - 1), \qquad a_t := \frac{h_t - 1}{\log_2 t}.$$

Define $L_n^{(t)}$ recursively as follows. For $1 \le n < tM$, set

$$L_n^{(t)} := A_n.$$

For $n \ge tM$, let $p_1, \dots, p_t$ be the balanced nondecreasing partition of n into t positive parts; this exists because $n \ge tM \ge t$. Set

$$q := \left\lfloor \frac{\lfloor n/t \rfloor}{M} \right\rfloor,$$

and define

$$L_n^{(t)} := L_{p_1}^{(t)} + \cdots + L_{p_t}^{(t)} + qB.$$

We claim that $H(n) \ge L_n^{(t)}$ for all $n \ge 1$. For $1 \le n < tM$, this is Theorem 3.3. Now let $n \ge tM$. Then $qM \le p_i$ for every i, so q copies of $\mathcal{L}_t$ form a legal frame on capacities $p_1, \dots, p_t$. By Corollary 2.6, applied recursively to witnesses for the smaller values $p_1, \dots, p_t$, we obtain a witness with $L_n^{(t)}$ vertices. Hence

$$H(n) \ge L_n^{(t)} \qquad \text{for all } n \ge 1.$$

We now prove by induction that there exists $C_t > 0$ such that

$$L_n^{(t)} \ge a_t n \log_2 n - C_t n \qquad \text{for all } n \ge 1.$$

Choose C_t large enough that this holds for all $1 \le n < tM$.

Assume now that $n \ge tM$ and that the claim is already true for smaller values. Then

$$L_n^{(t)} = L_{p_1}^{(t)} + \cdots + L_{p_t}^{(t)} + qB \ge a_t \sum_{i=1}^{t} p_i \log_2 p_i - C_t n + qB.$$

Since $x \mapsto x \log_2 x$ is convex and $\sum_i p_i = n$, the balanced partition minimizes the sum, so

$$\sum_{i=1}^{t} p_i \log_2 p_i \ge n \log_2(n/t) = n \log_2 n - n \log_2 t.$$

Also,

$$q = \left\lfloor \frac{\lfloor n/t \rfloor}{M} \right\rfloor \ge \frac{n}{tM} - 2,$$

and therefore

$$qB \geq (h_t - 1)n - 2Mt(h_t - 1).$$

Substituting these bounds,

$$\begin{aligned} L_n^{(t)} &\geq a_t\big(n \log_2 n - n \log_2 t\big) - C_t n + (h_t - 1)n - 2Mt(h_t - 1) \\ &= a_t\, n \log_2 n - C_t n - 2Mt(h_t - 1), \end{aligned}$$

because $a_t \log_2 t = h_t - 1$. After increasing C_t once more, the induction closes. Thus

$$H(n) \geq L_n^{(t)} \geq a_t\, n \log_2 n - C_t n \qquad \text{for all } n \geq 1.$$

□

Proof of Theorem 1.2. By Proposition 4.3,

$$k_n = \frac{1}{2} n \log_2 n + O(n).$$

Also $h_t = \ln t + O(1)$, so

$$\frac{2(h_t - 1)}{\log_2 t} \longrightarrow 2 \ln 2 \qquad (t \to \infty).$$

Fix $\varepsilon > 0$. Choose t so large that

$$\frac{2(h_t - 1)}{\log_2 t} > 2 \ln 2 - \varepsilon.$$

Then Theorem 4.4 gives

$$H(n) \geq \left(\frac{h_t - 1}{\log_2 t}\right) n \log_2 n - O_t(n).$$

Dividing by k_n and letting $n \to \infty$ yields

$$\liminf_{n \to \infty} \frac{H(n)}{k_n} \geq \frac{2(h_t - 1)}{\log_2 t} > 2 \ln 2 - \varepsilon.$$

Since $\varepsilon > 0$ was arbitrary, the theorem follows. □

Appendix A. Exact small frames used in the finite bootstrap

In this appendix every support pattern $\{i_1, \ldots, i_s\}$ is abbreviated by the concatenated string $i_1 \cdots i_s$, and coefficients denote multiplicity. These are exactly the support frames used in the rows marked $F_{(\ldots)}$ in Appendix C. They have all been precomputed and hard-coded into the final script, so the official constructor never solves any optimization problem at runtime.

Table 2: Exact small frames used below 60.

capacity vector	size	support multiset
$(2,2,2)$	5	12 + 13 + 23 + 2·123
$(4,4,6)$	11	3·12 + 13 + 23 + 6·123
$(6,6,6)$	15	3·12 + 3·13 + 3·23 + 6·123
$(2,2,2,3)$	9	12 + 13 + 23 + 123 + 124 + 134 + 234 + 2·1234
$(2,2,2,4)$	10	12 + 13 + 23 + 123 + 124 + 134 + 234 + 3·1234
$(4,4,6,9)$	22	3·12 + 13 + 23 + 3·123 + 2·124 + 3·134 + 2·234 + 7·1234
$(6,6,6,6)$	26	2·12 + 2·13 + 2·14 + 2·23 + 2·24 + 2·34 + 2·123 + 2·124 + 2·134 + 2·234 + 6·1234
$(6,6,6,7)$	26	3·12 + 2·13 + 14 + 2·23 + 24 + 2·34 + 123 + 2·124 + 2·134 + 3·234 + 7·1234
$(6,6,6,8)$	27	3·12 + 3·13 + 3·23 + 2·123 + 4·124 + 4·134 + 4·234 + 4·1234
$(6,6,6,9)$	28	3·12 + 3·13 + 3·23 + 2·123 + 4·124 + 4·134 + 4·234 + 5·1234
$(6,6,6,10)$	29	3·12 + 3·13 + 3·23 + 2·123 + 4·124 + 4·134 + 4·234 + 6·1234
$(2,2,2,3,4)$	15	12 + 13 + 23 + 124 + 125 + 134 + 135 + 145 + 2·234 + 2·2345 + 3·12345

Continued on next page.

Table 2 continued.

capacity vector	size	support multiset
$(3,4,4,4,6)$	25	$12+13+14+23+2{\cdot}34+45+123+2{\cdot}124+125+135+3{\cdot}345+1234+2{\cdot}1235+2{\cdot}1245+2345+4{\cdot}12345$
$(4,4,4,4,4)$	25	$12+13+14+15+23+24+25+34+35+45+234+235+245+345+2{\cdot}1234+2{\cdot}1235+2{\cdot}1245+2{\cdot}1345+2345+2{\cdot}12345$
$(4,4,4,4,6)$	27	$12+13+2{\cdot}14+23+24+34+2{\cdot}123+125+135+145+234+235+245+345+1234+1235+2{\cdot}1245+2{\cdot}1345+2{\cdot}2345+3{\cdot}12345$
$(5,6,6,6,6)$	35	$2{\cdot}12+13+14+15+2{\cdot}23+24+2{\cdot}34+35+45+2{\cdot}124+125+135+145+2{\cdot}245+3{\cdot}345+2{\cdot}1234+3{\cdot}1235+8{\cdot}12345$
$(6,6,6,6,6)$	38	$12+13+14+3{\cdot}15+23+24+25+34+35+45+2{\cdot}123+2{\cdot}124+2{\cdot}134+2{\cdot}234+2{\cdot}235+2{\cdot}245+2{\cdot}345+2{\cdot}1235+2{\cdot}1245+2{\cdot}1345+6{\cdot}12345$
$(6,6,6,6,10)$	42	$2{\cdot}12+2{\cdot}13+2{\cdot}14+2{\cdot}23+2{\cdot}24+2{\cdot}34+123+2{\cdot}124+125+134+135+145+234+2{\cdot}235+245+2{\cdot}345+1234+3{\cdot}1235+3{\cdot}1245+3{\cdot}1345+2{\cdot}2345+5{\cdot}12345$
$(6,6,6,6,11)$	42	$2{\cdot}12+2{\cdot}13+2{\cdot}14+2{\cdot}23+2{\cdot}24+2{\cdot}34+2{\cdot}123+2{\cdot}124+134+2{\cdot}234+5{\cdot}1235+5{\cdot}1245+6{\cdot}1345+5{\cdot}2345+2{\cdot}12345$
$(6,6,6,6,12)$	44	$2{\cdot}12+2{\cdot}13+2{\cdot}14+2{\cdot}23+2{\cdot}24+2{\cdot}34+2{\cdot}123+2{\cdot}124+2{\cdot}134+2{\cdot}234+6{\cdot}1235+6{\cdot}1245+6{\cdot}1345+6{\cdot}2345$
$(6,6,6,6,13)$	44	$2{\cdot}12+2{\cdot}13+2{\cdot}14+2{\cdot}23+2{\cdot}24+2{\cdot}34+2{\cdot}123+2{\cdot}124+2{\cdot}134+2{\cdot}234+6{\cdot}1235+5{\cdot}1245+6{\cdot}1345+5{\cdot}2345+2{\cdot}12345$

Appendix B. Explicit boosters and four-way residue gadgets

The next table lists the seven higher-arity boosters used in the finite bootstrap. They are the only gadgets in the paper living on more than five blocks.

Table 3: Explicit boosters.

gadget	capacity vector	size	support multiset
B_{15}	$(2,2,2,2,2,2,3)$	22	$13+15+24+26+35+46+1237+1245+1267+1346+1567+2347+2356+3457+4567+12457+13467+23567+2{\cdot}123456+2{\cdot}1234567$
B_{16}	$(2,2,2,2,2,2,2,2)$	26	$15+26+37+48+127+134+168+238+245+356+467+578+12346+12358+12478+13678+14567+23457+25678+34568+123567+124568+134578+234678+2{\cdot}12345678$
B_{17}	$(2,2,2,3,4,4)$	22	$12+13+23+124+125+134+234+236+245+256+345+456+1236+1246+1346+2{\cdot}1356+5{\cdot}123456$
B_{19}	$(2,2,2,3,4,6)$	24	$12+13+23+45+124+134+234+2{\cdot}1235+1245+1345+3{\cdot}1456+2345+3{\cdot}12346+3{\cdot}12356+3{\cdot}23456$
B_{31}	$(3,4,4,4,4,4,4)$	50	$23+28+34+45+56+67+78+124+125+126+127+135+136+137+138+146+147+148+157+158+168+235+248+267+346+378+457+568+23467+23567+23568+24568+24578+34578+34678+2{\cdot}1234567+2{\cdot}1234568+2{\cdot}1234578+2{\cdot}1234678+2{\cdot}1235678+2{\cdot}1245678+2{\cdot}1345678+2345678$

Continued on next page.

Table 3 continued.

gadget	capacity vector	size	support multiset
B_{32}	$(4,4,4,4,4,4,4,4)$	53	$13 + 2{\cdot}15 + 17 + 24 + 2{\cdot}26 + 28 + 35 + 2{\cdot}37 + 46 + 2{\cdot}48 + 57 + 68 + 1234 + 1238 + 1247 + 1256 + 1278 + 1346 + 2{\cdot}1357 + 1368 + 1458 + 1467 + 1678 + 2345 + 2358 + 2367 + 2457 + 2{\cdot}2468 + 2578 + 3456 + 3478 + 3568 + 4567 + 5678 + 1234567 + 1234568 + 1234578 + 1234678 + 1235678 + 1245678 + 1345678 + 2345678 + 5{\cdot}12345678$
B_{33}	$(1,4,4,4,4,4,4,4,4)$	55	$26 + 37 + 48 + 59 + 124 + 128 + 135 + 139 + 146 + 157 + 168 + 179 + 1247 + 1257 + 1258 + 1358 + 1368 + 1369 + 1469 + 1479 + 2345 + 2349 + 2356 + 2378 + 2389 + 2459 + 2468 + 2679 + 2789 + 3456 + 3467 + 3489 + 3579 + 4567 + 4578 + 5678 + 5689 + 6789 + 12367 + 12569 + 13478 + 14589 + 12345678 + 12345679 + 12345689 + 12345789 + 12346789 + 12356789 + 12456789 + 13456789 + 4{\cdot}23456789 + 123456789$

The balanced four-way induction uses the following residue gadgets.

Table 4: Four-way residue gadgets R_r.

r	residual capacity vector	size	support multiset
0	$(0,0,0,0)$	0	$\varnothing$
1	$(0,0,0,1)$	0	$\varnothing$
2	$(0,0,1,1)$	1	34
3	$(0,1,1,1)$	2	$2{\cdot}234$
4	$(1,1,1,1)$	4	$123 + 124 + 134 + 234$
5	$(1,1,1,2)$	4	$123 + 3{\cdot}1234$
6	$(1,1,2,2)$	6	$12 + 34 + 123 + 124 + 2{\cdot}1234$
7	$(1,2,2,2)$	6	$23 + 24 + 34 + 123 + 2{\cdot}1234$
8	$(2,2,2,2)$	8	$2{\cdot}123 + 2{\cdot}124 + 2{\cdot}134 + 2{\cdot}234$
9	$(2,2,2,3)$	9	$12 + 13 + 23 + 123 + 124 + 134 + 234 + 2{\cdot}1234$
10	$(2,2,3,3)$	10	$2{\cdot}123 + 2{\cdot}124 + 134 + 234 + 12 + 34 + 2{\cdot}1234$
11	$(2,3,3,3)$	10	$2{\cdot}123 + 124 + 134 + 234 + 23 + 24 + 34 + 2{\cdot}1234$

Appendix C. The finite bootstrap table

Appendix C records the exact recursive choice used for each $2 \le n < 60$. The notation is as follows:

- $\mathrm{Bin}(a,b)$ denotes the classical binary construction on capacities (a,b);
- $F_{(n_1,\dots,n_t)}$ denotes the exact frame from Appendix A;
- $Q(a,b,c,d)$ denotes the balanced four-way construction with the classical 13-vertex core and the appropriate residue gadget;
- $B_{15}, B_{16}, B_{17}, B_{19}, B_{31}, B_{32}, B_{33}$ are the explicit boosters from Appendix B.

The fifth column is the slack

$$\Delta_n := 25A_n - 26k_n.$$

We emphasize that the theorem only requires $\Delta_n \ge 0$ for $15 \le n < 60$; the values for $2 \le n < 15$ are included only to make the recursive constructor fully explicit from the base case.

Table 5: Finite bootstrap data for the recursive construction.

n	construction	k_n	A_n	Δ_n
2	$\mathrm{Bin}(1,1)$	3	3	-3
3	$\mathrm{Bin}(1,2)$	5	5	-5
4	$\mathrm{Bin}(2,2)$	8	8	-8
5	$\mathrm{Bin}(2,3)$	10	10	-10
6	$F_{(2,2,2)}$	13	14	12
7	$\mathrm{Bin}(3,4)$	16	16	-16
8	$\mathrm{Bin}(4,4)$	20	20	-20
9	$F_{(2,2,2,3)}$	22	23	3
10	$F_{(2,2,2,4)}$	25	27	25
11	$\mathrm{Bin}(5,6)$	28	29	-3
12	$\mathrm{Bin}(6,6)$	32	34	18
13	$F_{(2,2,2,3,4)}$	35	37	15
14	$F_{(4,4,6)}$	39	41	11
15	B_{15}	43	45	7
16	B_{16}	48	50	2
17	B_{17}	50	52	0
18	$F_{(6,6,6)}$	53	57	47
19	B_{19}	56	60	44
20	$F_{(4,4,4,4,4)}$	60	65	65
21	$F_{(3,4,4,4,6)}$	63	68	62
22	$F_{(4,4,4,4,6)}$	67	73	83
23	$F_{(4,4,6,9)}$	71	75	29
24	$F_{(6,6,6,6)}$	76	82	74
25	$F_{(6,6,6,7)}$	79	84	46
26	$F_{(6,6,6,8)}$	83	89	67
27	$F_{(6,6,6,9)}$	87	93	63
28	$F_{(6,6,6,10)}$	92	98	58
29	$F_{(5,6,6,6,6)}$	96	101	29
30	$F_{(6,6,6,6,6)}$	101	108	74
31	B_{31}	106	111	19
32	B_{32}	112	117	13
33	B_{33}	114	120	36
34	$F_{(6,6,6,6,10)}$	117	125	83
35	$F_{(6,6,6,6,11)}$	120	127	55
36	$F_{(6,6,6,6,12)}$	124	134	126
37	$F_{(6,6,6,6,13)}$	127	137	123
38	$Q(9,9,10,10)$	131	140	94
39	$Q(9,10,10,10)$	135	145	115
40	$Q(10,10,10,10)$	140	151	135
41	$\mathrm{Bin}(20,21)$	143	153	107
42	$\mathrm{Bin}(21,21)$	147	157	103
43	$\mathrm{Bin}(21,22)$	151	162	124
44	$\mathrm{Bin}(22,22)$	156	168	144
45	$\mathrm{Bin}(22,23)$	160	170	90
46	$Q(11,11,12,12)$	165	175	85
47	$\mathrm{Bin}(23,24)$	170	180	80
48	$\mathrm{Bin}(24,24)$	176	188	124
49	$Q(12,12,12,13)$	179	191	121
50	$Q(12,12,13,13)$	183	195	117
51	$Q(12,13,13,13)$	187	199	113
52	$\mathrm{Bin}(26,26)$	192	204	108
53	$\mathrm{Bin}(26,27)$	196	208	104
54	$Q(13,13,14,14)$	201	214	124

Continued on next page.

Table 5 continued.

n	construction	k_n	A_n	Δ_n
55	$\mathrm{Bin}(27, 28)$	206	218	94
56	$\mathrm{Bin}(28, 28)$	212	224	88
57	$Q(14, 14, 14, 15)$	216	229	109
58	$Q(14, 14, 15, 15)$	221	234	104
59	$\mathrm{Bin}(29, 30)$	226	238	74

Appendix D. Verifier-facing Python constructor

The following is the official script required by the problem statement, reproduced verbatim so that the present source is self-contained. It defines a single function `solution(n: int) -> str`, contains no file-level executable code, uses only the Python standard library, and outputs the witness hypergraph in the requested edge-string format.

Program 1. Pure-Python constructor witnessing $H(n) > \frac{26}{25}k_n$ for every $n > 15$.

```
def solution(n: int) -> str:
    from functools import lru_cache

    if n < 1:
        raise ValueError("n must be positive")

    EXACT_FRAMES = {(2, 2, 2): [(0, 1), (0, 2), (1, 2), (0, 1, 2), (0, 1, 2)],
 (4, 4, 6): [(0, 1),
             (0, 1),
             (0, 1),
             (0, 2),
             (1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2)],
 (6, 6, 6): [(0, 1),
             (0, 1),
             (0, 1),
             (0, 2),
             (0, 2),
             (0, 2),
             (1, 2),
             (1, 2),
             (1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2),
             (0, 1, 2)],
 (2, 2, 2, 3): [(0, 1),
                (0, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
```

```
                (0, 1, 2, 3)],
(2, 2, 2, 4): [(0, 1),
                (0, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(4, 4, 6, 9): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 6): [(0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 3),
                (0, 3),
                (1, 2),
                (1, 2),
                (1, 3),
                (1, 3),
                (2, 3),
                (2, 3),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
```

```
                (0, 1, 2, 3)],
(6, 6, 6, 7): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 3),
                (1, 2),
                (1, 2),
                (1, 3),
                (2, 3),
                (2, 3),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 8): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 2),
                (1, 2),
                (1, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 9): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
```

```
                (0, 2),
                (0, 2),
                (1, 2),
                (1, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 10): [(0, 1),
                 (0, 1),
                 (0, 1),
                 (0, 2),
                 (0, 2),
                 (0, 2),
                 (1, 2),
                 (1, 2),
                 (1, 2),
                 (0, 1, 2),
                 (0, 1, 2),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3)],
(2, 2, 2, 3, 4): [(0, 1),
                   (0, 2),
                   (1, 2),
                   (0, 1, 3),
                   (0, 1, 4),
```

```
                    (0, 2, 3),
                    (0, 2, 4),
                    (0, 3, 4),
                    (1, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(3, 4, 4, 4, 6): [(0, 1),
                    (0, 2),
                    (0, 3),
                    (1, 2),
                    (2, 3),
                    (2, 3),
                    (3, 4),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 1, 4),
                    (0, 2, 4),
                    (2, 3, 4),
                    (2, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(4, 4, 4, 4, 4): [(0, 1),
                    (0, 2),
                    (0, 3),
                    (0, 4),
                    (1, 2),
                    (1, 3),
                    (1, 4),
                    (2, 3),
                    (2, 4),
                    (3, 4),
                    (1, 2, 3),
                    (1, 2, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 3),
                    (0, 1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
```

```
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4)],
(4, 4, 4, 4, 6): [(0, 1),
                   (0, 2),
                   (0, 3),
                   (0, 3),
                   (1, 2),
                   (1, 3),
                   (2, 3),
                   (0, 1, 2),
                   (0, 1, 2),
                   (0, 1, 4),
                   (0, 2, 4),
                   (0, 3, 4),
                   (1, 2, 3),
                   (1, 2, 4),
                   (1, 3, 4),
                   (2, 3, 4),
                   (0, 1, 2, 3),
                   (0, 1, 2, 4),
                   (0, 1, 3, 4),
                   (0, 1, 3, 4),
                   (0, 2, 3, 4),
                   (0, 2, 3, 4),
                   (1, 2, 3, 4),
                   (1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4)],
(5, 6, 6, 6, 6): [(0, 1),
                   (0, 1),
                   (0, 2),
                   (0, 3),
                   (0, 4),
                   (1, 2),
                   (1, 2),
                   (1, 3),
                   (2, 3),
                   (2, 3),
                   (2, 4),
                   (3, 4),
                   (0, 1, 3),
                   (0, 1, 3),
                   (0, 1, 4),
                   (0, 2, 4),
                   (0, 3, 4),
                   (1, 3, 4),
                   (1, 3, 4),
                   (2, 3, 4),
                   (2, 3, 4),
                   (2, 3, 4),
                   (0, 1, 2, 3),
                   (0, 1, 2, 3),
                   (0, 1, 2, 4),
                   (0, 1, 2, 4),
                   (0, 1, 2, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
```

```
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 6): [(0, 1),
                    (0, 2),
                    (0, 3),
                    (0, 4),
                    (0, 4),
                    (0, 4),
                    (1, 2),
                    (1, 3),
                    (1, 4),
                    (2, 3),
                    (2, 4),
                    (3, 4),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (1, 2, 4),
                    (1, 2, 4),
                    (1, 3, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 10): [(0, 1),
                     (0, 1),
                     (0, 2),
                     (0, 2),
                     (0, 3),
                     (0, 3),
                     (1, 2),
                     (1, 2),
                     (1, 3),
                     (1, 3),
                     (2, 3),
                     (2, 3),
                     (0, 1, 2),
                     (0, 1, 3),
```

```
                     (0, 1, 3),
                     (0, 1, 4),
                     (0, 2, 3),
                     (0, 2, 4),
                     (0, 3, 4),
                     (1, 2, 3),
                     (1, 2, 4),
                     (1, 2, 4),
                     (1, 3, 4),
                     (2, 3, 4),
                     (2, 3, 4),
                     (0, 1, 2, 3),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 2, 3, 4),
                     (0, 2, 3, 4),
                     (0, 2, 3, 4),
                     (1, 2, 3, 4),
                     (1, 2, 3, 4),
                     (0, 1, 2, 3, 4),
                     (0, 1, 2, 3, 4),
                     (0, 1, 2, 3, 4),
                     (0, 1, 2, 3, 4),
                     (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 11): [(0, 1),
                     (0, 1),
                     (0, 2),
                     (0, 2),
                     (0, 3),
                     (0, 3),
                     (1, 2),
                     (1, 2),
                     (1, 3),
                     (1, 3),
                     (2, 3),
                     (2, 3),
                     (0, 1, 2),
                     (0, 1, 2),
                     (0, 1, 3),
                     (0, 1, 3),
                     (0, 2, 3),
                     (1, 2, 3),
                     (1, 2, 3),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 2, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 1, 3, 4),
                     (0, 2, 3, 4),
```

```
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 12): [(0, 1),
                    (0, 1),
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4)],
(6, 6, 6, 6, 13): [(0, 1),
                    (0, 1),
```

```
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)]}

BOOSTERS = {'boost15': ((2, 2, 2, 2, 2, 2, 3),
         [(0, 2),
          (0, 4),
          (1, 3),
          (1, 5),
          (2, 4),
          (3, 5),
          (0, 1, 2, 6),
          (0, 1, 3, 4),
          (0, 1, 5, 6),
          (0, 2, 3, 5),
          (0, 4, 5, 6),
          (1, 2, 3, 6),
          (1, 2, 4, 5),
          (2, 3, 4, 6),
```

```
             (3, 4, 5, 6),
             (0, 1, 3, 4, 6),
             (0, 2, 3, 5, 6),
             (1, 2, 4, 5, 6),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5, 6),
             (0, 1, 2, 3, 4, 5, 6)]),
'boost16': ((2, 2, 2, 2, 2, 2, 2, 2),
            [(0, 4),
             (1, 5),
             (2, 6),
             (3, 7),
             (0, 1, 6),
             (0, 2, 3),
             (0, 5, 7),
             (1, 2, 7),
             (1, 3, 4),
             (2, 4, 5),
             (3, 5, 6),
             (4, 6, 7),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 4, 7),
             (0, 1, 3, 6, 7),
             (0, 2, 5, 6, 7),
             (0, 3, 4, 5, 6),
             (1, 2, 3, 4, 6),
             (1, 4, 5, 6, 7),
             (2, 3, 4, 5, 7),
             (0, 1, 2, 4, 5, 6),
             (0, 1, 3, 4, 5, 7),
             (0, 2, 3, 4, 6, 7),
             (1, 2, 3, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7)]),
'boost17': ((2, 2, 2, 3, 4, 4),
            [(0, 1),
             (0, 2),
             (1, 2),
             (0, 1, 3),
             (0, 1, 4),
             (0, 2, 3),
             (1, 2, 3),
             (1, 2, 5),
             (1, 3, 4),
             (1, 4, 5),
             (2, 3, 4),
             (3, 4, 5),
             (0, 1, 2, 5),
             (0, 1, 3, 5),
             (0, 2, 3, 5),
             (0, 2, 4, 5),
             (0, 2, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5)]),
```

```
'boost19': ((2, 2, 2, 3, 4, 6),
            [(0, 1),
             (0, 2),
             (1, 2),
             (3, 4),
             (0, 1, 3),
             (0, 2, 3),
             (1, 2, 3),
             (0, 1, 2, 4),
             (0, 1, 2, 4),
             (0, 1, 3, 4),
             (0, 2, 3, 4),
             (0, 3, 4, 5),
             (0, 3, 4, 5),
             (0, 3, 4, 5),
             (1, 2, 3, 4),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 4, 5),
             (0, 1, 2, 4, 5),
             (0, 1, 2, 4, 5),
             (1, 2, 3, 4, 5),
             (1, 2, 3, 4, 5),
             (1, 2, 3, 4, 5)]),
'boost31': ((3, 4, 4, 4, 4, 4, 4, 4),
            [(1, 2),
             (1, 7),
             (2, 3),
             (3, 4),
             (4, 5),
             (5, 6),
             (6, 7),
             (0, 1, 3),
             (0, 1, 4),
             (0, 1, 5),
             (0, 1, 6),
             (0, 2, 4),
             (0, 2, 5),
             (0, 2, 6),
             (0, 2, 7),
             (0, 3, 5),
             (0, 3, 6),
             (0, 3, 7),
             (0, 4, 6),
             (0, 4, 7),
             (0, 5, 7),
             (1, 2, 4),
             (1, 3, 7),
             (1, 5, 6),
             (2, 3, 5),
             (2, 6, 7),
             (3, 4, 6),
             (4, 5, 7),
             (1, 2, 3, 5, 6),
             (1, 2, 4, 5, 6),
             (1, 2, 4, 5, 7),
             (1, 3, 4, 5, 7),
```

```
              (1, 3, 4, 6, 7),
              (2, 3, 4, 6, 7),
              (2, 3, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 7),
              (0, 1, 2, 3, 4, 5, 7),
              (0, 1, 2, 3, 4, 6, 7),
              (0, 1, 2, 3, 4, 6, 7),
              (0, 1, 2, 3, 5, 6, 7),
              (0, 1, 2, 3, 5, 6, 7),
              (0, 1, 2, 4, 5, 6, 7),
              (0, 1, 2, 4, 5, 6, 7),
              (0, 1, 3, 4, 5, 6, 7),
              (0, 1, 3, 4, 5, 6, 7),
              (0, 2, 3, 4, 5, 6, 7),
              (0, 2, 3, 4, 5, 6, 7),
              (1, 2, 3, 4, 5, 6, 7)]),
'boost32': ((4, 4, 4, 4, 4, 4, 4, 4),
             [(0, 2),
              (0, 4),
              (0, 4),
              (0, 6),
              (1, 3),
              (1, 5),
              (1, 5),
              (1, 7),
              (2, 4),
              (2, 6),
              (2, 6),
              (3, 5),
              (3, 7),
              (3, 7),
              (4, 6),
              (5, 7),
              (0, 1, 2, 3),
              (0, 1, 2, 7),
              (0, 1, 3, 6),
              (0, 1, 4, 5),
              (0, 1, 6, 7),
              (0, 2, 3, 5),
              (0, 2, 4, 6),
              (0, 2, 4, 6),
              (0, 2, 5, 7),
              (0, 3, 4, 7),
              (0, 3, 5, 6),
              (0, 5, 6, 7),
              (1, 2, 3, 4),
              (1, 2, 4, 7),
              (1, 2, 5, 6),
              (1, 3, 4, 6),
              (1, 3, 5, 7),
              (1, 3, 5, 7),
              (1, 4, 6, 7),
              (2, 3, 4, 5),
              (2, 3, 6, 7),
              (2, 4, 5, 7),
              (3, 4, 5, 6),
```

```
              (4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 7),
              (0, 1, 2, 3, 4, 6, 7),
              (0, 1, 2, 3, 5, 6, 7),
              (0, 1, 2, 4, 5, 6, 7),
              (0, 1, 3, 4, 5, 6, 7),
              (0, 2, 3, 4, 5, 6, 7),
              (1, 2, 3, 4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6, 7)]),
 'boost33': ((1, 4, 4, 4, 4, 4, 4, 4, 4),
             [(1, 5),
              (2, 6),
              (3, 7),
              (4, 8),
              (0, 1, 3),
              (0, 1, 7),
              (0, 2, 4),
              (0, 2, 8),
              (0, 3, 5),
              (0, 4, 6),
              (0, 5, 7),
              (0, 6, 8),
              (0, 1, 3, 6),
              (0, 1, 4, 6),
              (0, 1, 4, 7),
              (0, 2, 4, 7),
              (0, 2, 5, 7),
              (0, 2, 5, 8),
              (0, 3, 5, 8),
              (0, 3, 6, 8),
              (1, 2, 3, 4),
              (1, 2, 3, 8),
              (1, 2, 4, 5),
              (1, 2, 6, 7),
              (1, 2, 7, 8),
              (1, 3, 4, 8),
              (1, 3, 5, 7),
              (1, 5, 6, 8),
              (1, 6, 7, 8),
              (2, 3, 4, 5),
              (2, 3, 5, 6),
              (2, 3, 7, 8),
              (2, 4, 6, 8),
              (3, 4, 5, 6),
              (3, 4, 6, 7),
              (4, 5, 6, 7),
              (4, 5, 7, 8),
              (5, 6, 7, 8),
              (0, 1, 2, 5, 6),
              (0, 1, 4, 5, 8),
              (0, 2, 3, 6, 7),
              (0, 3, 4, 7, 8),
              (0, 1, 2, 3, 4, 5, 6, 7),
```

```
             (0, 1, 2, 3, 4, 5, 6, 8),
             (0, 1, 2, 3, 4, 5, 7, 8),
             (0, 1, 2, 3, 4, 6, 7, 8),
             (0, 1, 2, 3, 5, 6, 7, 8),
             (0, 1, 2, 4, 5, 6, 7, 8),
             (0, 1, 3, 4, 5, 6, 7, 8),
             (0, 2, 3, 4, 5, 6, 7, 8),
             (1, 2, 3, 4, 5, 6, 7, 8),
             (1, 2, 3, 4, 5, 6, 7, 8),
             (1, 2, 3, 4, 5, 6, 7, 8),
             (1, 2, 3, 4, 5, 6, 7, 8),
             (0, 1, 2, 3, 4, 5, 6, 7, 8)])}

   CORE4 = ((0, 1),
(0, 2),
(0, 3),
(1, 2),
(1, 3),
(2, 3),
(0, 1, 2),
(0, 1, 3),
(0, 2, 3),
(1, 2, 3),
(0, 1, 2, 3),
(0, 1, 2, 3),
(0, 1, 2, 3))

   RESIDUAL4 = {0: (),
1: (),
2: ((2, 3),),
3: ((1, 2, 3), (1, 2, 3)),
4: ((0, 1, 2), (0, 1, 3), (0, 2, 3), (1, 2, 3)),
5: ((0, 1, 2), (0, 1, 2, 3), (0, 1, 2, 3), (0, 1, 2, 3)),
6: ((0, 1), (2, 3), (0, 1, 2), (0, 1, 3), (0, 1, 2, 3), (0, 1, 2, 3)),
7: ((1, 2), (1, 3), (2, 3), (0, 1, 2), (0, 1, 2, 3), (0, 1, 2, 3)),
8: ((0, 1, 2),
    (0, 1, 3),
    (0, 2, 3),
    (1, 2, 3),
    (0, 1, 2),
    (0, 1, 3),
    (0, 2, 3),
    (1, 2, 3)),
9: ((0, 1),
    (0, 2),
    (1, 2),
    (0, 1, 2),
    (0, 1, 3),
    (0, 2, 3),
    (1, 2, 3),
    (0, 1, 2, 3),
    (0, 1, 2, 3)),
10: ((0, 1, 2),
     (0, 1, 3),
     (0, 2, 3),
     (1, 2, 3),
     (0, 1),
     (2, 3),
```

```
     (0, 1, 2),
     (0, 1, 3),
     (0, 1, 2, 3),
     (0, 1, 2, 3)),
11: ((0, 1, 2),
     (0, 1, 3),
     (0, 2, 3),
     (1, 2, 3),
     (1, 2),
     (1, 3),
     (2, 3),
     (0, 1, 2),
     (0, 1, 2, 3),
     (0, 1, 2, 3))}

   CHOICE_UNDER_60 = {1: ('base', (), 0),
2: ('bin', (1, 1), 1),
3: ('bin', (1, 2), 1),
4: ('bin', (2, 2), 2),
5: ('bin', (2, 3), 2),
6: ('frame', (2, 2, 2), 5),
7: ('bin', (3, 4), 3),
8: ('bin', (4, 4), 4),
9: ('frame', (2, 2, 2, 3), 9),
10: ('frame', (2, 2, 2, 4), 10),
11: ('bin', (5, 6), 5),
12: ('bin', (6, 6), 6),
13: ('frame', (2, 2, 2, 3, 4), 15),
14: ('frame', (4, 4, 6), 11),
15: ('boost15', (2, 2, 2, 2, 2, 2, 3), 22),
16: ('boost16', (2, 2, 2, 2, 2, 2, 2, 2), 26),
17: ('boost17', (2, 2, 2, 3, 4, 4), 22),
18: ('frame', (6, 6, 6), 15),
19: ('boost19', (2, 2, 2, 3, 4, 6), 24),
20: ('frame', (4, 4, 4, 4, 4), 25),
21: ('frame', (3, 4, 4, 4, 6), 25),
22: ('frame', (4, 4, 4, 4, 6), 27),
23: ('frame', (4, 4, 6, 9), 22),
24: ('frame', (6, 6, 6, 6), 26),
25: ('frame', (6, 6, 6, 7), 26),
26: ('frame', (6, 6, 6, 8), 27),
27: ('frame', (6, 6, 6, 9), 28),
28: ('frame', (6, 6, 6, 10), 29),
29: ('frame', (5, 6, 6, 6, 6), 35),
30: ('frame', (6, 6, 6, 6, 6), 38),
31: ('boost31', (3, 4, 4, 4, 4, 4, 4, 4), 50),
32: ('boost32', (4, 4, 4, 4, 4, 4, 4, 4), 53),
33: ('boost33', (1, 4, 4, 4, 4, 4, 4, 4, 4), 55),
34: ('frame', (6, 6, 6, 6, 10), 42),
35: ('frame', (6, 6, 6, 6, 11), 42),
36: ('frame', (6, 6, 6, 6, 12), 44),
37: ('frame', (6, 6, 6, 6, 13), 44),
38: ('quadres', (9, 9, 10, 10), 40),
39: ('quadres', (9, 10, 10, 10), 41),
40: ('quadres', (10, 10, 10, 10), 43),
41: ('bin', (20, 21), 20),
42: ('bin', (21, 21), 21),
43: ('bin', (21, 22), 21),
```

```
44: ('bin', (22, 22), 22),
45: ('bin', (22, 23), 22),
46: ('quadres', (11, 11, 12, 12), 49),
47: ('bin', (23, 24), 23),
48: ('bin', (24, 24), 24),
49: ('quadres', (12, 12, 12, 13), 52),
50: ('quadres', (12, 12, 13, 13), 53),
51: ('quadres', (12, 13, 13, 13), 54),
52: ('bin', (26, 26), 26),
53: ('bin', (26, 27), 26),
54: ('quadres', (13, 13, 14, 14), 58),
55: ('bin', (27, 28), 27),
56: ('bin', (28, 28), 28),
57: ('quadres', (14, 14, 14, 15), 61),
58: ('quadres', (14, 14, 15, 15), 62),
59: ('bin', (29, 30), 29)}

def support_mask(length: int, offset: int) -> int:
    return ((1 << length) - 1) << offset

def balanced_four_parts(m: int):
    parts = [m // 4] * 4
    for i in range(m % 4):
        parts[3 - i] += 1
    return tuple(parts)

def choice_for_n(m: int):
    if m < 60:
        return CHOICE_UNDER_60[m]
    parts = balanced_four_parts(m)
    return ("quad4", parts, 13 * (m // 12) + len(RESIDUAL4[m % 12]))

def gadget_for_choice(choice):
    kind, parts, bonus = choice
    if kind == "base":
        return ()
    if kind == "bin":
        a, b = parts
        return ((0, 1),) * min(a, b)
    if kind == "frame":
        gadget = EXACT_FRAMES[tuple(parts)]
    elif kind in ("quad4", "quadres"):
        q, r = divmod(sum(parts), 12)
        gadget = CORE4 * q + RESIDUAL4[r]
    elif kind in BOOSTERS:
        hard_parts, gadget = BOOSTERS[kind]
        if tuple(parts) != tuple(hard_parts):
            raise RuntimeError("booster part mismatch")
    else:
        raise RuntimeError(f"unknown choice kind: {kind}")
    if len(gadget) != bonus:
        raise RuntimeError("bonus mismatch")
    return gadget

@lru_cache(maxsize=None)
def build(m: int):
    choice = choice_for_n(m)
    kind = choice[0]
```

```
        if kind == "base":
            return (1,), 1

        parts = tuple(choice[1])
        children = [build(p) for p in parts]
        gadget = gadget_for_choice(choice)

        vertex_masks = []
        block_masks = []
        offset = 0
        for p, (child_vertices, child_edges) in zip(parts, children):
            if child_edges != p:
                raise RuntimeError("edge count mismatch in recursion")
            for mask in child_vertices:
                vertex_masks.append(mask << offset)
            block_masks.append(support_mask(p, offset))
            offset += p

        for S in gadget:
            mask = 0
            for i in S:
                mask |= block_masks[i]
            vertex_masks.append(mask)

        return tuple(vertex_masks), offset

    vertex_masks, edge_count = build(n)
    edges = [[] for _ in range(edge_count)]
    for label, mask in enumerate(vertex_masks, start=1):
        x = mask
        while x:
            lb = x & -x
            e = lb.bit_length() - 1
            edges[e].append(label)
            x ^= lb

    return ",".join("{" + ",".join(str(v) for v in edge) + "}" for edge in edges)
```

Appendix E. Verification script

The finite checks used in Proposition 3.1 and Proposition 3.6 are carried out by the following standalone script `verify_hn_26_25.py`, reproduced verbatim so that the present source is self-contained. It verifies:

- every frame inequality $\omega_{\mathcal{F}}(T, I) \leq \sum_{j \in T \setminus I} n_j$ for every explicit gadget $\mathcal{F}$ used (exact frames, boosters, and residue gadgets);
- internal consistency of the bootstrap recursion below 60 (the stated A_n matches the recursion induced by the choice table);
- the bootstrap-window inequalities $25A_n \geq 26k_n$ for $15 \leq n < 60$;
- spot checks of the asymptotic four-way branch and the supporting arithmetic.

To run the verifier, execute `python3 verify_hn_26_25.py`. It prints `All checks passed.`

Program 2. Verifier for the gadget-frame checks and bootstrap inequalities.

```
from functools import lru_cache

EXACT_FRAMES = {(2, 2, 2): [(0, 1), (0, 2), (1, 2), (0, 1, 2), (0, 1, 2)],
 (4, 4, 6): [(0, 1),
             (0, 1),
```

```
            (0, 1),
            (0, 2),
            (1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2)],
(6, 6, 6): [(0, 1),
            (0, 1),
            (0, 1),
            (0, 2),
            (0, 2),
            (0, 2),
            (1, 2),
            (1, 2),
            (1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2),
            (0, 1, 2)],
(2, 2, 2, 3): [(0, 1),
               (0, 2),
               (1, 2),
               (0, 1, 2),
               (0, 1, 3),
               (0, 2, 3),
               (1, 2, 3),
               (0, 1, 2, 3),
               (0, 1, 2, 3)],
(2, 2, 2, 4): [(0, 1),
               (0, 2),
               (1, 2),
               (0, 1, 2),
               (0, 1, 3),
               (0, 2, 3),
               (1, 2, 3),
               (0, 1, 2, 3),
               (0, 1, 2, 3),
               (0, 1, 2, 3)],
(4, 4, 6, 9): [(0, 1),
               (0, 1),
               (0, 1),
               (0, 2),
               (1, 2),
               (0, 1, 2),
               (0, 1, 2),
               (0, 1, 2),
               (0, 1, 3),
               (0, 1, 3),
               (0, 2, 3),
               (0, 2, 3),
               (0, 2, 3),
               (1, 2, 3),
               (1, 2, 3),
```

```
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 6): [(0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 3),
                (0, 3),
                (1, 2),
                (1, 2),
                (1, 3),
                (1, 3),
                (2, 3),
                (2, 3),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 7): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 3),
                (1, 2),
                (1, 2),
                (1, 3),
                (2, 3),
                (2, 3),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
```

```
                (0, 1, 2, 3)],
(6, 6, 6, 8): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 2),
                (1, 2),
                (1, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 9): [(0, 1),
                (0, 1),
                (0, 1),
                (0, 2),
                (0, 2),
                (0, 2),
                (1, 2),
                (1, 2),
                (1, 2),
                (0, 1, 2),
                (0, 1, 2),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 1, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (0, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3),
                (0, 1, 2, 3)],
(6, 6, 6, 10): [(0, 1),
                 (0, 1),
```

```
                 (0, 1),
                 (0, 2),
                 (0, 2),
                 (0, 2),
                 (1, 2),
                 (1, 2),
                 (1, 2),
                 (0, 1, 2),
                 (0, 1, 2),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 1, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (0, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3),
                 (0, 1, 2, 3)],
(2, 2, 2, 3, 4): [(0, 1),
                  (0, 2),
                  (1, 2),
                  (0, 1, 3),
                  (0, 1, 4),
                  (0, 2, 3),
                  (0, 2, 4),
                  (0, 3, 4),
                  (1, 2, 3),
                  (1, 2, 3),
                  (1, 2, 3, 4),
                  (1, 2, 3, 4),
                  (0, 1, 2, 3, 4),
                  (0, 1, 2, 3, 4),
                  (0, 1, 2, 3, 4)],
(3, 4, 4, 4, 6): [(0, 1),
                  (0, 2),
                  (0, 3),
                  (1, 2),
                  (2, 3),
                  (2, 3),
                  (3, 4),
                  (0, 1, 2),
                  (0, 1, 3),
                  (0, 1, 3),
                  (0, 1, 4),
                  (0, 2, 4),
                  (2, 3, 4),
                  (2, 3, 4),
                  (2, 3, 4),
                  (0, 1, 2, 3),
```

```
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(4, 4, 4, 4, 4): [(0, 1),
                    (0, 2),
                    (0, 3),
                    (0, 4),
                    (1, 2),
                    (1, 3),
                    (1, 4),
                    (2, 3),
                    (2, 4),
                    (3, 4),
                    (1, 2, 3),
                    (1, 2, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 3),
                    (0, 1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(4, 4, 4, 4, 6): [(0, 1),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 4),
                    (0, 2, 4),
                    (0, 3, 4),
                    (1, 2, 3),
                    (1, 2, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
```

```
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4)],
(5, 6, 6, 6, 6): [(0, 1),
                   (0, 1),
                   (0, 2),
                   (0, 3),
                   (0, 4),
                   (1, 2),
                   (1, 2),
                   (1, 3),
                   (2, 3),
                   (2, 3),
                   (2, 4),
                   (3, 4),
                   (0, 1, 3),
                   (0, 1, 3),
                   (0, 1, 4),
                   (0, 2, 4),
                   (0, 3, 4),
                   (1, 3, 4),
                   (1, 3, 4),
                   (2, 3, 4),
                   (2, 3, 4),
                   (2, 3, 4),
                   (0, 1, 2, 3),
                   (0, 1, 2, 3),
                   (0, 1, 2, 4),
                   (0, 1, 2, 4),
                   (0, 1, 2, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4),
                   (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 6): [(0, 1),
                   (0, 2),
                   (0, 3),
                   (0, 4),
                   (0, 4),
                   (0, 4),
                   (1, 2),
                   (1, 3),
                   (1, 4),
                   (2, 3),
                   (2, 4),
                   (3, 4),
                   (0, 1, 2),
                   (0, 1, 2),
                   (0, 1, 3),
                   (0, 1, 3),
                   (0, 2, 3),
                   (0, 2, 3),
                   (1, 2, 3),
                   (1, 2, 3),
```

```
                    (1, 2, 4),
                    (1, 2, 4),
                    (1, 3, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 10): [(0, 1),
                    (0, 1),
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 1, 4),
                    (0, 2, 3),
                    (0, 2, 4),
                    (0, 3, 4),
                    (1, 2, 3),
                    (1, 2, 4),
                    (1, 2, 4),
                    (1, 3, 4),
                    (2, 3, 4),
                    (2, 3, 4),
                    (0, 1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
```

```
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 11): [(0, 1),
                    (0, 1),
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)],
(6, 6, 6, 6, 12): [(0, 1),
                    (0, 1),
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
```

```
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4)],
(6, 6, 6, 6, 13): [(0, 1),
                    (0, 1),
                    (0, 2),
                    (0, 2),
                    (0, 3),
                    (0, 3),
                    (1, 2),
                    (1, 2),
                    (1, 3),
                    (1, 3),
                    (2, 3),
                    (2, 3),
                    (0, 1, 2),
                    (0, 1, 2),
                    (0, 1, 3),
                    (0, 1, 3),
                    (0, 2, 3),
                    (0, 2, 3),
                    (1, 2, 3),
                    (1, 2, 3),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 2, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
```

```
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 1, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (0, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (1, 2, 3, 4),
                    (0, 1, 2, 3, 4),
                    (0, 1, 2, 3, 4)]}

BOOSTERS = {'boost15': ((2, 2, 2, 2, 2, 2, 3),
             [(0, 2),
              (0, 4),
              (1, 3),
              (1, 5),
              (2, 4),
              (3, 5),
              (0, 1, 2, 6),
              (0, 1, 3, 4),
              (0, 1, 5, 6),
              (0, 2, 3, 5),
              (0, 4, 5, 6),
              (1, 2, 3, 6),
              (1, 2, 4, 5),
              (2, 3, 4, 6),
              (3, 4, 5, 6),
              (0, 1, 3, 4, 6),
              (0, 2, 3, 5, 6),
              (1, 2, 4, 5, 6),
              (0, 1, 2, 3, 4, 5),
              (0, 1, 2, 3, 4, 5),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 6)]),
 'boost16': ((2, 2, 2, 2, 2, 2, 2, 2),
             [(0, 4),
              (1, 5),
              (2, 6),
              (3, 7),
              (0, 1, 6),
              (0, 2, 3),
              (0, 5, 7),
              (1, 2, 7),
              (1, 3, 4),
              (2, 4, 5),
              (3, 5, 6),
              (4, 6, 7),
              (0, 1, 2, 3, 5),
              (0, 1, 2, 4, 7),
              (0, 1, 3, 6, 7),
              (0, 2, 5, 6, 7),
              (0, 3, 4, 5, 6),
```

```
             (1, 2, 3, 4, 6),
             (1, 4, 5, 6, 7),
             (2, 3, 4, 5, 7),
             (0, 1, 2, 4, 5, 6),
             (0, 1, 3, 4, 5, 7),
             (0, 2, 3, 4, 6, 7),
             (1, 2, 3, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7)]),
'boost17': ((2, 2, 2, 3, 4, 4),
            [(0, 1),
             (0, 2),
             (1, 2),
             (0, 1, 3),
             (0, 1, 4),
             (0, 2, 3),
             (1, 2, 3),
             (1, 2, 5),
             (1, 3, 4),
             (1, 4, 5),
             (2, 3, 4),
             (3, 4, 5),
             (0, 1, 2, 5),
             (0, 1, 3, 5),
             (0, 2, 3, 5),
             (0, 2, 4, 5),
             (0, 2, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5),
             (0, 1, 2, 3, 4, 5)]),
'boost19': ((2, 2, 2, 3, 4, 6),
            [(0, 1),
             (0, 2),
             (1, 2),
             (3, 4),
             (0, 1, 3),
             (0, 2, 3),
             (1, 2, 3),
             (0, 1, 2, 4),
             (0, 1, 2, 4),
             (0, 1, 3, 4),
             (0, 2, 3, 4),
             (0, 3, 4, 5),
             (0, 3, 4, 5),
             (0, 3, 4, 5),
             (1, 2, 3, 4),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 3, 5),
             (0, 1, 2, 4, 5),
             (0, 1, 2, 4, 5),
             (0, 1, 2, 4, 5),
             (1, 2, 3, 4, 5),
             (1, 2, 3, 4, 5),
             (1, 2, 3, 4, 5)]),
'boost31': ((3, 4, 4, 4, 4, 4, 4, 4),
```

```
             [(1, 2),
              (1, 7),
              (2, 3),
              (3, 4),
              (4, 5),
              (5, 6),
              (6, 7),
              (0, 1, 3),
              (0, 1, 4),
              (0, 1, 5),
              (0, 1, 6),
              (0, 2, 4),
              (0, 2, 5),
              (0, 2, 6),
              (0, 2, 7),
              (0, 3, 5),
              (0, 3, 6),
              (0, 3, 7),
              (0, 4, 6),
              (0, 4, 7),
              (0, 5, 7),
              (1, 2, 4),
              (1, 3, 7),
              (1, 5, 6),
              (2, 3, 5),
              (2, 6, 7),
              (3, 4, 6),
              (4, 5, 7),
              (1, 2, 3, 5, 6),
              (1, 2, 4, 5, 6),
              (1, 2, 4, 5, 7),
              (1, 3, 4, 5, 7),
              (1, 3, 4, 6, 7),
              (2, 3, 4, 6, 7),
              (2, 3, 5, 6, 7),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 6),
              (0, 1, 2, 3, 4, 5, 7),
              (0, 1, 2, 3, 4, 5, 7),
              (0, 1, 2, 3, 4, 6, 7),
              (0, 1, 2, 3, 4, 6, 7),
              (0, 1, 2, 3, 5, 6, 7),
              (0, 1, 2, 3, 5, 6, 7),
              (0, 1, 2, 4, 5, 6, 7),
              (0, 1, 2, 4, 5, 6, 7),
              (0, 1, 3, 4, 5, 6, 7),
              (0, 1, 3, 4, 5, 6, 7),
              (0, 2, 3, 4, 5, 6, 7),
              (0, 2, 3, 4, 5, 6, 7),
              (1, 2, 3, 4, 5, 6, 7)]),
 'boost32': ((4, 4, 4, 4, 4, 4, 4, 4),
             [(0, 2),
              (0, 4),
              (0, 4),
              (0, 6),
              (1, 3),
              (1, 5),
              (1, 5),
```

```
             (1, 7),
             (2, 4),
             (2, 6),
             (2, 6),
             (3, 5),
             (3, 7),
             (3, 7),
             (4, 6),
             (5, 7),
             (0, 1, 2, 3),
             (0, 1, 2, 7),
             (0, 1, 3, 6),
             (0, 1, 4, 5),
             (0, 1, 6, 7),
             (0, 2, 3, 5),
             (0, 2, 4, 6),
             (0, 2, 4, 6),
             (0, 2, 5, 7),
             (0, 3, 4, 7),
             (0, 3, 5, 6),
             (0, 5, 6, 7),
             (1, 2, 3, 4),
             (1, 2, 4, 7),
             (1, 2, 5, 6),
             (1, 3, 4, 6),
             (1, 3, 5, 7),
             (1, 3, 5, 7),
             (1, 4, 6, 7),
             (2, 3, 4, 5),
             (2, 3, 6, 7),
             (2, 4, 5, 7),
             (3, 4, 5, 6),
             (4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6),
             (0, 1, 2, 3, 4, 5, 7),
             (0, 1, 2, 3, 4, 6, 7),
             (0, 1, 2, 3, 5, 6, 7),
             (0, 1, 2, 4, 5, 6, 7),
             (0, 1, 3, 4, 5, 6, 7),
             (0, 2, 3, 4, 5, 6, 7),
             (1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7),
             (0, 1, 2, 3, 4, 5, 6, 7)]),
'boost33': ((1, 4, 4, 4, 4, 4, 4, 4, 4),
            [(1, 5),
             (2, 6),
             (3, 7),
             (4, 8),
             (0, 1, 3),
             (0, 1, 7),
             (0, 2, 4),
             (0, 2, 8),
             (0, 3, 5),
             (0, 4, 6),
             (0, 5, 7),
```

```
                (0, 6, 8),
                (0, 1, 3, 6),
                (0, 1, 4, 6),
                (0, 1, 4, 7),
                (0, 2, 4, 7),
                (0, 2, 5, 7),
                (0, 2, 5, 8),
                (0, 3, 5, 8),
                (0, 3, 6, 8),
                (1, 2, 3, 4),
                (1, 2, 3, 8),
                (1, 2, 4, 5),
                (1, 2, 6, 7),
                (1, 2, 7, 8),
                (1, 3, 4, 8),
                (1, 3, 5, 7),
                (1, 5, 6, 8),
                (1, 6, 7, 8),
                (2, 3, 4, 5),
                (2, 3, 5, 6),
                (2, 3, 7, 8),
                (2, 4, 6, 8),
                (3, 4, 5, 6),
                (3, 4, 6, 7),
                (4, 5, 6, 7),
                (4, 5, 7, 8),
                (5, 6, 7, 8),
                (0, 1, 2, 5, 6),
                (0, 1, 4, 5, 8),
                (0, 2, 3, 6, 7),
                (0, 3, 4, 7, 8),
                (0, 1, 2, 3, 4, 5, 6, 7),
                (0, 1, 2, 3, 4, 5, 6, 8),
                (0, 1, 2, 3, 4, 5, 7, 8),
                (0, 1, 2, 3, 4, 6, 7, 8),
                (0, 1, 2, 3, 5, 6, 7, 8),
                (0, 1, 2, 4, 5, 6, 7, 8),
                (0, 1, 3, 4, 5, 6, 7, 8),
                (0, 2, 3, 4, 5, 6, 7, 8),
                (1, 2, 3, 4, 5, 6, 7, 8),
                (1, 2, 3, 4, 5, 6, 7, 8),
                (1, 2, 3, 4, 5, 6, 7, 8),
                (1, 2, 3, 4, 5, 6, 7, 8),
                (0, 1, 2, 3, 4, 5, 6, 7, 8)])}

RESIDUAL4 = {0: ((0, 0, 0, 0), []),
 1: ((0, 0, 0, 1), []),
 2: ((0, 0, 1, 1), [(2, 3)]),
 3: ((0, 1, 1, 1), [(1, 2, 3), (1, 2, 3)]),
 4: ((1, 1, 1, 1), [(0, 1, 2), (0, 1, 3), (0, 2, 3), (1, 2, 3)]),
 5: ((1, 1, 1, 2), [(0, 1, 2), (0, 1, 2, 3), (0, 1, 2, 3), (0, 1, 2, 3)]),
 6: ((1, 1, 2, 2), [(0, 1), (2, 3), (0, 1, 2), (0, 1, 3), (0, 1, 2, 3), (0, 1, 2, 3)])
    ,
 7: ((1, 2, 2, 2), [(1, 2), (1, 3), (2, 3), (0, 1, 2), (0, 1, 2, 3), (0, 1, 2, 3)]),
 8: ((2, 2, 2, 2),
     [(0, 1, 2), (0, 1, 3), (0, 2, 3), (1, 2, 3), (0, 1, 2), (0, 1, 3), (0, 2, 3), (1,
     2, 3)]),
 9: ((2, 2, 2, 3),
```

```
     [(0, 1),
      (0, 2),
      (1, 2),
      (0, 1, 2),
      (0, 1, 3),
      (0, 2, 3),
      (1, 2, 3),
      (0, 1, 2, 3),
      (0, 1, 2, 3)]),
 10: ((2, 2, 3, 3),
      [(0, 1, 2),
       (0, 1, 3),
       (0, 2, 3),
       (1, 2, 3),
       (0, 1),
       (2, 3),
       (0, 1, 2),
       (0, 1, 3),
       (0, 1, 2, 3),
       (0, 1, 2, 3)]),
 11: ((2, 3, 3, 3),
      [(0, 1, 2),
       (0, 1, 3),
       (0, 2, 3),
       (1, 2, 3),
       (1, 2),
       (1, 3),
       (2, 3),
       (0, 1, 2),
       (0, 1, 2, 3),
       (0, 1, 2, 3)])}

A_UNDER_60 = (0,
 1,
 3,
 5,
 8,
 10,
 14,
 16,
 20,
 23,
 27,
 29,
 34,
 37,
 41,
 45,
 50,
 52,
 57,
 60,
 65,
 68,
 73,
 75,
 82,
 84,
```

```
 89,
 93,
 98,
 101,
 108,
 111,
 117,
 120,
 125,
 127,
 134,
 137,
 140,
 145,
 151,
 153,
 157,
 162,
 168,
 170,
 175,
 180,
 188,
 191,
 195,
 199,
 204,
 208,
 214,
 218,
 224,
 229,
 234,
 238)

CHOICE_UNDER_60 = {1: ('base', (), 0),
 2: ('bin', (1, 1), 1),
 3: ('bin', (1, 2), 1),
 4: ('bin', (2, 2), 2),
 5: ('bin', (2, 3), 2),
 6: ('frame', (2, 2, 2), 5),
 7: ('bin', (3, 4), 3),
 8: ('bin', (4, 4), 4),
 9: ('frame', (2, 2, 2, 3), 9),
 10: ('frame', (2, 2, 2, 4), 10),
 11: ('bin', (5, 6), 5),
 12: ('bin', (6, 6), 6),
 13: ('frame', (2, 2, 2, 3, 4), 15),
 14: ('frame', (4, 4, 6), 11),
 15: ('boost15', (2, 2, 2, 2, 2, 2, 3), 22),
 16: ('boost16', (2, 2, 2, 2, 2, 2, 2, 2), 26),
 17: ('boost17', (2, 2, 2, 3, 4, 4), 22),
 18: ('frame', (6, 6, 6), 15),
 19: ('boost19', (2, 2, 2, 3, 4, 6), 24),
 20: ('frame', (4, 4, 4, 4, 4), 25),
 21: ('frame', (3, 4, 4, 4, 6), 25),
 22: ('frame', (4, 4, 4, 4, 6), 27),
 23: ('frame', (4, 4, 6, 9), 22),
```

```
 24: ('frame', (6, 6, 6, 6), 26),
 25: ('frame', (6, 6, 6, 7), 26),
 26: ('frame', (6, 6, 6, 8), 27),
 27: ('frame', (6, 6, 6, 9), 28),
 28: ('frame', (6, 6, 6, 10), 29),
 29: ('frame', (5, 6, 6, 6, 6), 35),
 30: ('frame', (6, 6, 6, 6, 6), 38),
 31: ('boost31', (3, 4, 4, 4, 4, 4, 4, 4), 50),
 32: ('boost32', (4, 4, 4, 4, 4, 4, 4, 4), 53),
 33: ('boost33', (1, 4, 4, 4, 4, 4, 4, 4, 4), 55),
 34: ('frame', (6, 6, 6, 6, 10), 42),
 35: ('frame', (6, 6, 6, 6, 11), 42),
 36: ('frame', (6, 6, 6, 6, 12), 44),
 37: ('frame', (6, 6, 6, 6, 13), 44),
 38: ('quadres', (9, 9, 10, 10), 40),
 39: ('quadres', (9, 10, 10, 10), 41),
 40: ('quadres', (10, 10, 10, 10), 43),
 41: ('bin', (20, 21), 20),
 42: ('bin', (21, 21), 21),
 43: ('bin', (21, 22), 21),
 44: ('bin', (22, 22), 22),
 45: ('bin', (22, 23), 22),
 46: ('quadres', (11, 11, 12, 12), 49),
 47: ('bin', (23, 24), 23),
 48: ('bin', (24, 24), 24),
 49: ('quadres', (12, 12, 12, 13), 52),
 50: ('quadres', (12, 12, 13, 13), 53),
 51: ('quadres', (12, 13, 13, 13), 54),
 52: ('bin', (26, 26), 26),
 53: ('bin', (26, 27), 26),
 54: ('quadres', (13, 13, 14, 14), 58),
 55: ('bin', (27, 28), 27),
 56: ('bin', (28, 28), 28),
 57: ('quadres', (14, 14, 14, 15), 61),
 58: ('quadres', (14, 14, 15, 15), 62),
 59: ('bin', (29, 30), 29)}

def omega(gadget, T, I):
    T = set(T)
    I = set(I)
    return sum(1 for S in gadget if set(S).issubset(T) and len(set(S) & I) == 1)

def is_frame(gadget, cap):
    t = len(cap)
    for Tmask in range(1 << t):
        T = [i for i in range(t) if (Tmask >> i) & 1]
        for Imask in range(1 << t):
            if Imask & ~Tmask:
                continue
            I = [i for i in range(t) if (Imask >> i) & 1]
            lhs = omega(gadget, T, I)
            rhs = sum(cap[j] for j in T if j not in I)
            if lhs > rhs:
                return False, (tuple(T), tuple(I), lhs, rhs)
    return True, None

def k_values(N):
    k = [0] * (N + 1)
```

```
    k[1] = 1
    for n in range(2, N + 1):
        a = n // 2
        b = (n + 1) // 2
        k[n] = a + k[a] + k[b]
    return k

def e0(m): return (13 * m) // 3
def e1(m): return (13 * m + 1) // 3
def e2(m): return (13 * m + 5) // 3
def e3(m): return (13 * m + 6) // 3

def A_value(n):
    if n < 60:
        return A_UNDER_60[n]
    parts = [n // 4] * 4
    for i in range(n % 4):
        parts[3 - i] += 1
    return sum(A_value(p) for p in parts) + 13 * (n // 12) + len(RESIDUAL4[n % 12][1])

def main():
    print("Checking exact frames...")
    for parts, gadget in EXACT_FRAMES.items():
        ok, cert = is_frame(gadget, parts)
        print(parts, ok, cert)
        assert ok

    print("Checking boosters...")
    for name, (parts, gadget) in BOOSTERS.items():
        ok, cert = is_frame(gadget, parts)
        print(name, ok, cert)
        assert ok

    print("Checking four-way residues...")
    for r, (cap, gadget) in sorted(RESIDUAL4.items()):
        ok, cert = is_frame(gadget, cap)
        print(r, ok, cert)
        assert ok

    print("Checking finite table and bound...")
    k = k_values(59)
    for n in range(2, 60):
        kind, parts, bonus = CHOICE_UNDER_60[n]
        value = sum(A_UNDER_60[p] for p in parts) + bonus if kind != "base" else 1
        assert value == A_UNDER_60[n], (n, value, A_UNDER_60[n])
    for n in range(15, 60):
        assert 25 * A_UNDER_60[n] >= 26 * k[n], (n, A_UNDER_60[n], k[n])

    print("Checking induction inequalities...")
    for m in range(15, 10000):
        assert 25 * e0(m) >= 26 * (4 * m)
        assert 25 * e1(m) >= 26 * (4 * m)
        assert 25 * e2(m) >= 26 * (4 * m + 1)
        assert 25 * e3(m) >= 26 * (4 * m + 2)

    print("Spot-checking global bound up to 500...")
    k = k_values(500)
    for n in range(15, 501):
```

```
        assert 25 * A_value(n) >= 26 * k[n], (n, A_value(n), k[n])

    print("All checks passed.")

if __name__ == "__main__":
    main()
```

References

[1] Will Brian and Paul B. Larson, *Choosing between incompatible ideals*, European Journal of Combinatorics **96** (2021), Article 103349. doi:10.1016/j.ejc.2021.103349.

[2] Epoch AI, *A Ramsey-style Problem on Hypergraphs*, FrontierMath open problem page. https://epoch.ai/frontiermath/open-problems/ramsey-hypergraphs/. Accessed 10 March 2026.